



Full-Stack Software Development

JPA Advanced

R. Naudts, J. Pieck, J. Rombouts, B. Van Impe

CONTENTS

- Transactions
- Entity Lifecycle
- Cascades
- Fetch Types
- Orphan Removal
- Conclusions

Transactions

Entity Lifecycle

Cascades

Fetch Types

Orphan Removal

Conclusions

Transactions

TRANSACTIONS

- What is a transaction?

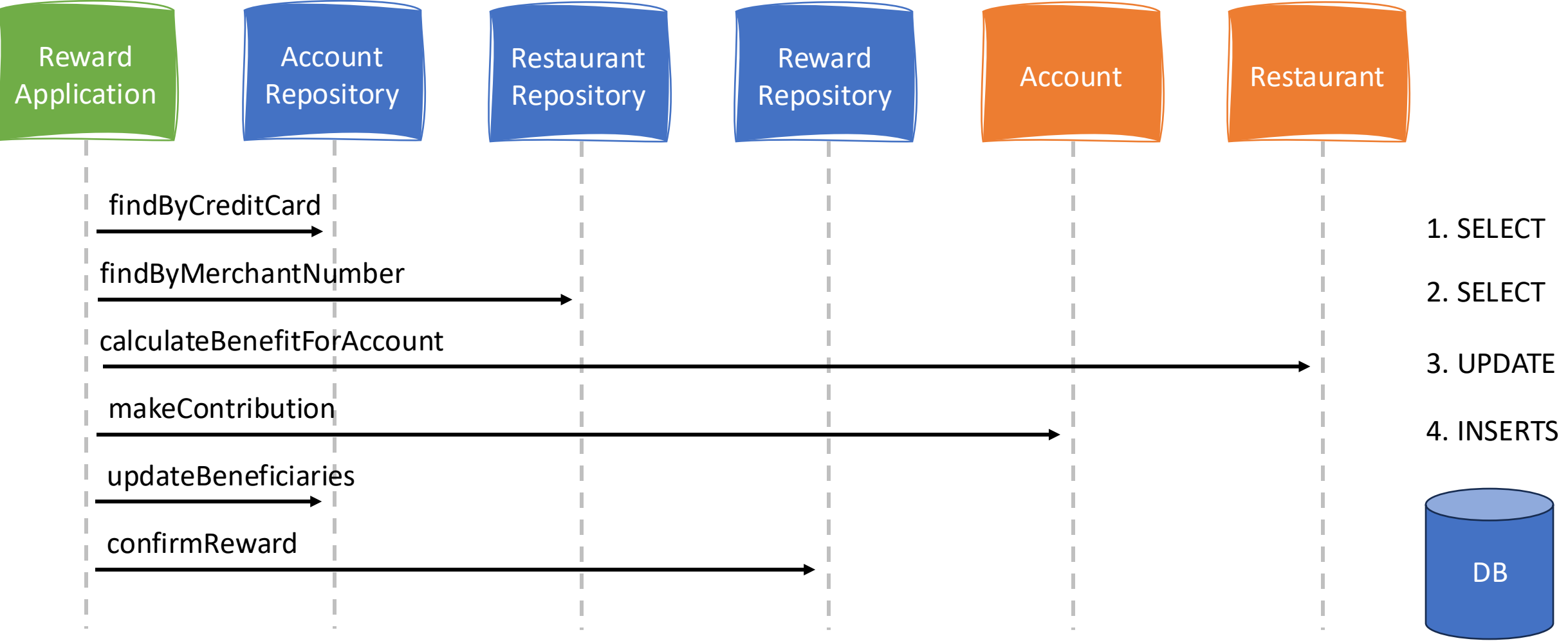
A set of tasks which take place as a single, atomic, consistent, isolated, durable operation.

TRANSACTIONS

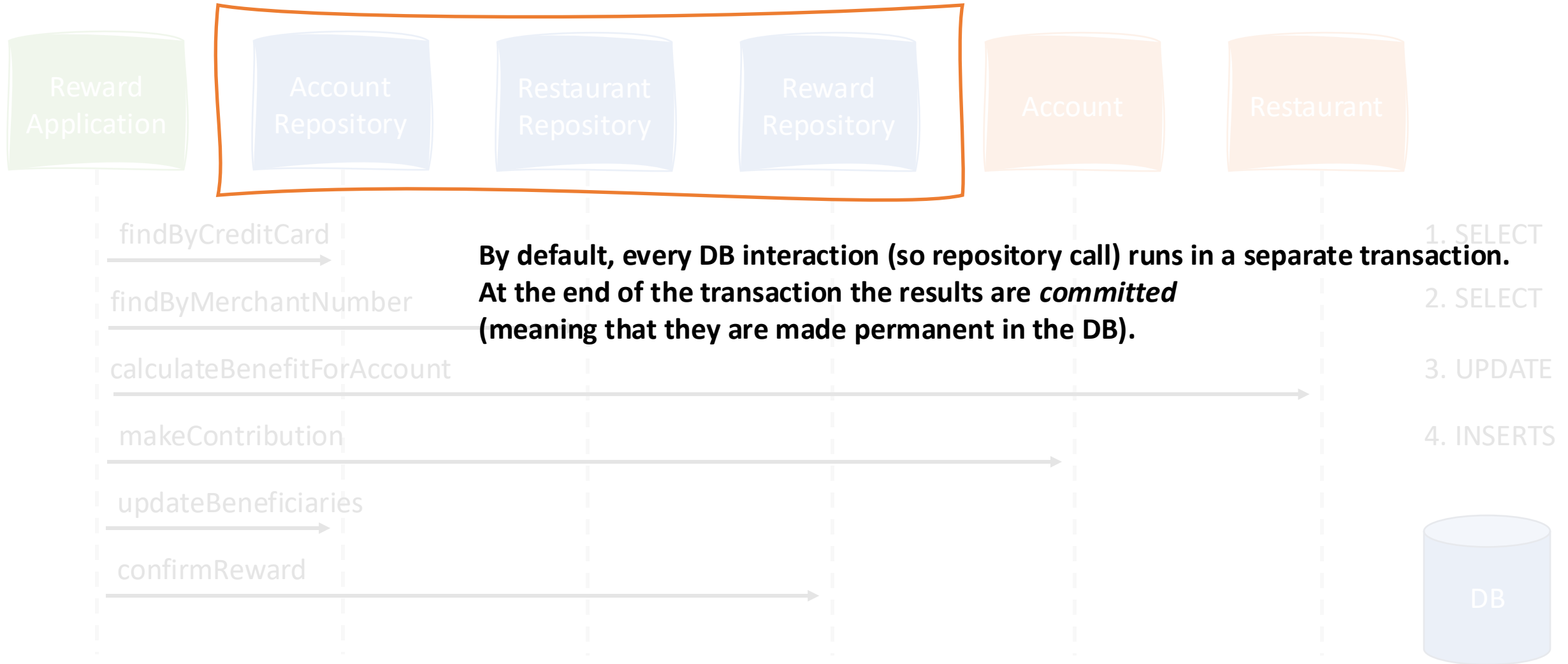
A set of tasks which take place as a single, **a**tomic, **c**onsistent, **i**solated, **d**urable operation.

- We say that transactions enforce the **ACID**-principles
 - Atomic: each unit-ofwork is an all-or-nothing operation
 - Consistent: integrity constraints are never violated
 - Isolated: transactions are isolated from each other
 - Durable: committed changes are permitted

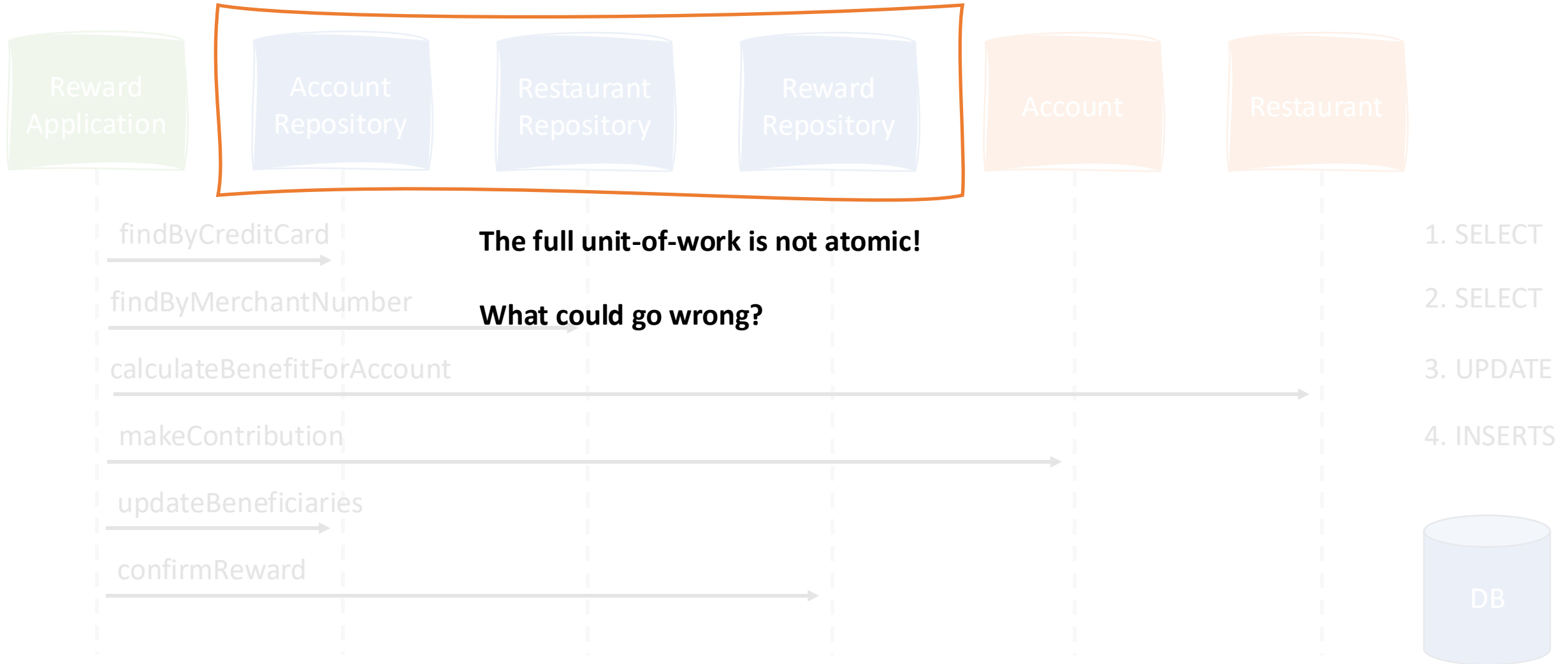
TRANSACTIONS



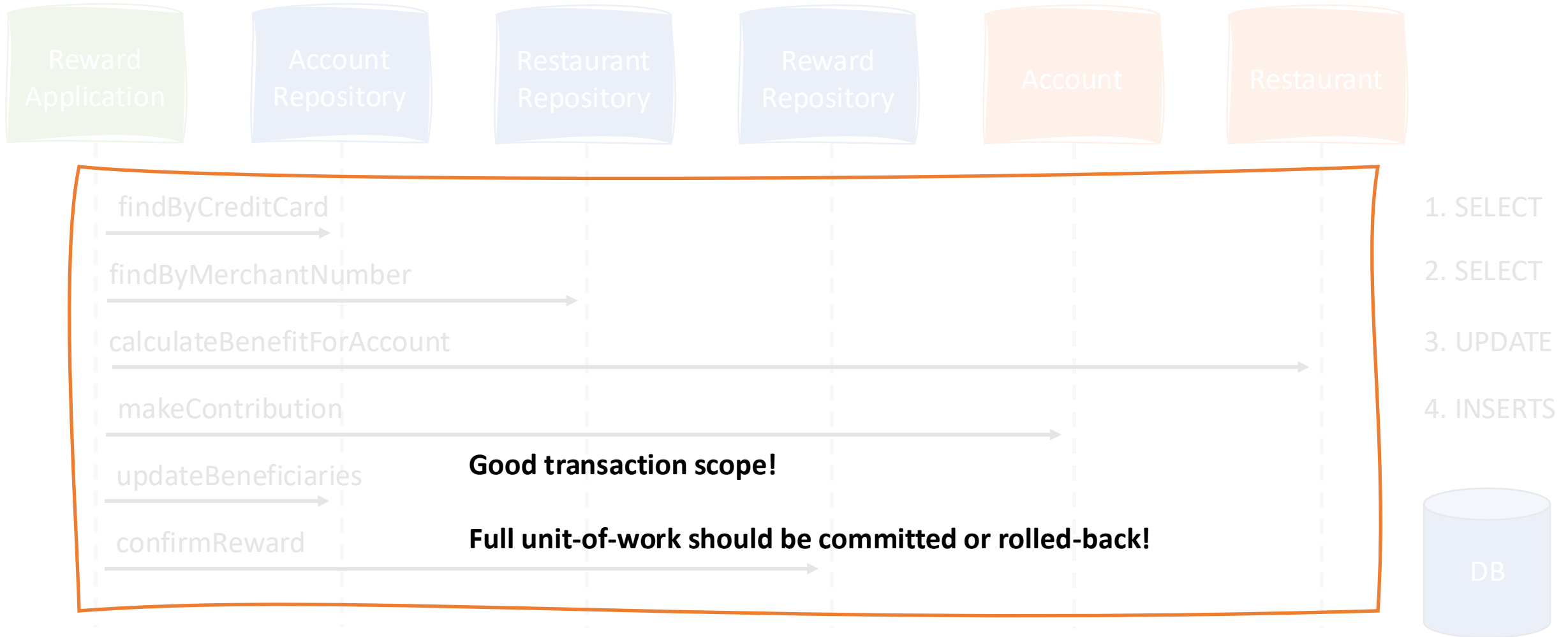
TRANSACTIONS



TRANSACTIONS



TRANSACTIONS



TRANSACTIONS

```
@Transactional
public Lecturer addLecturer(LecturerInput lecturerInput) {
    var user = userService.signup(Role.LECTURER, lecturerInput.user());
    return lecturerRepository.save(new Lecturer(lecturerInput.expertise(), user));
}
```

- In Spring, you can add `@Transactional` to wrap all code from the start to the end of the annotated method in a transaction
 - The `@Transactional` only works if your method is called from another class!
- In this example, a lecturer and user will always be committed together or not at all

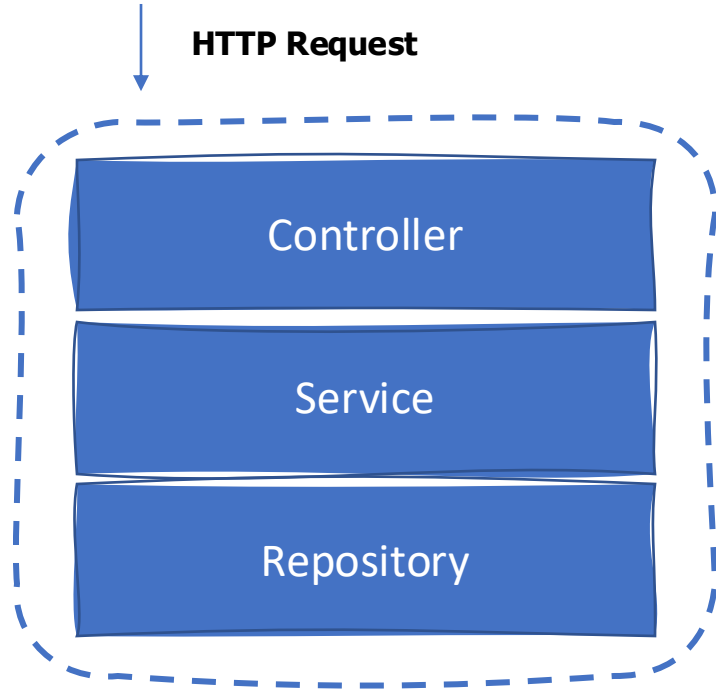
TRANSACTIONS

```
@Service
@Transactional
public class LecturerService {

    // ...
}
```

- Placing **@Transactional** at class-level applies it to all public methods in the class
 - You can use **@Transactional** at method-level to overwrite settings from the class-level one

TRANSACTIONS



Backend

Translates HTTP request to Java and translates Java back to a HTTP response

Guards application rules and gives commands to repositories to store, retrieve data

Communicates with the database and provides an API to use the database from within Java code

- It is recommended to use service-level operations as the units-of-work in a layered Spring application
 - Service methods act as orchestrators that govern the repositories and business logic
 - All services should be annotated with `@Transactional`
 - Service methods should be designed with the transactions in mind

TRANSACTIONS

```
@Transactional
public Lecturer addLecturer(LecturerInput lecturerInput) {
    var user = userService.signup(Role.LECTURER, lecturerInput.user());

    throw new RuntimeException();

    return lecturerRepository.save(new Lecturer(lecturerInput.expertise(), user));
}
```

- A transaction does not commit (all temporary changes made to the database are *rolled-back*) if a RuntimeException is thrown in the wrapped code
- In the example above, nothing is committed, and the user saved in the user table disappears again

TRANSACTIONS

`@Transactional(noRollbackFor = AnUnimportantException.class)`

- You can override this behaviour using the `rollbackFor` and `noRollbackFor` attributes

TRANSACTIONS

```
@PostConstruct
public void init() {
    transactionTemplate.executeWithoutResult((transactionStatus) -> {
        clearAll();

        final var fullStack = courseRepository.save(new Course(
            "Full-stack development",
            "Learn how to build a full stack web application.",
            2,
            6
        ));

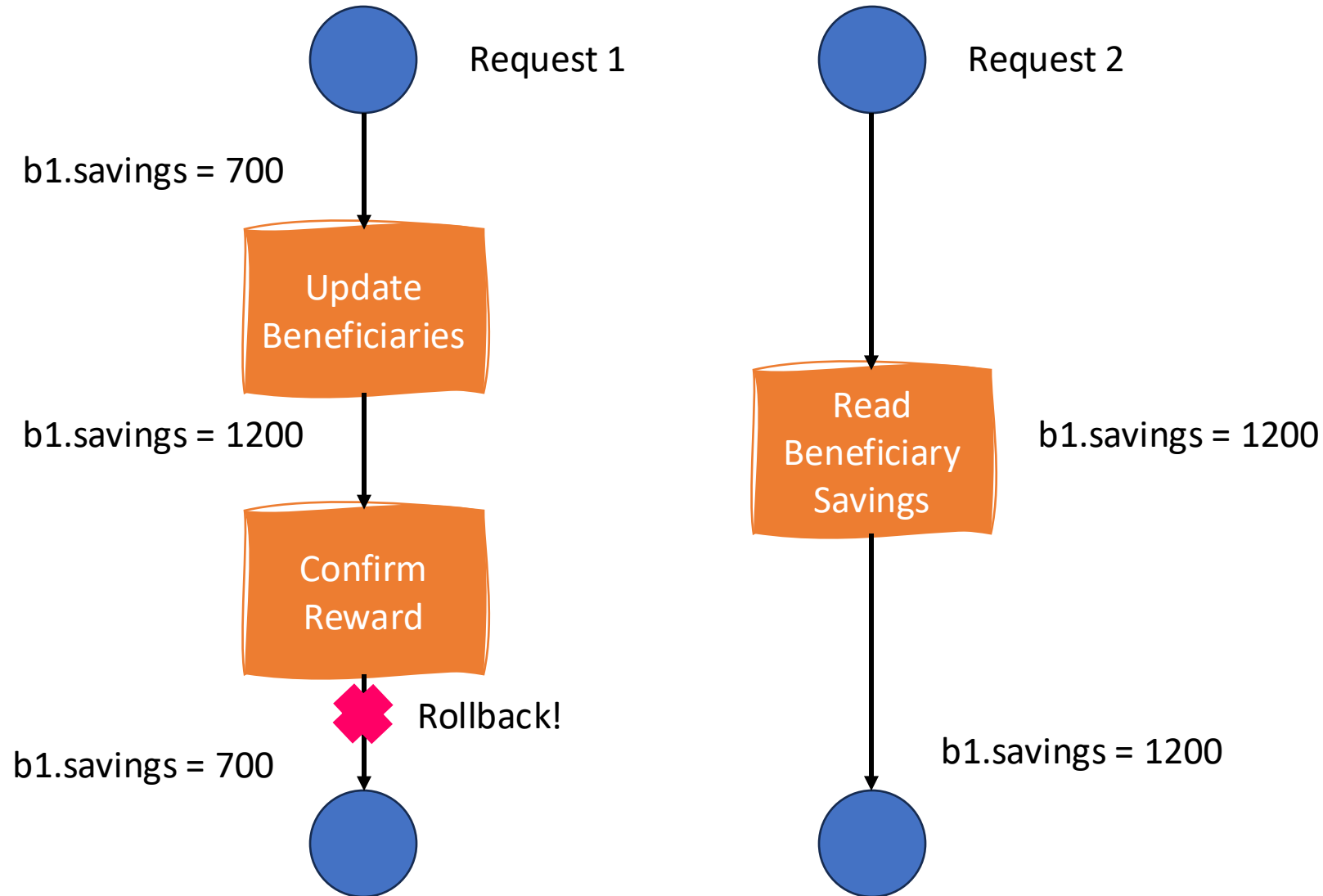
        // ...
    }
}
```

- There are places where it is not possible to use `@Transactional` due to technical reasons, like the `PostConstruct` method from the `DbInitializer`
- In this case, you can use a *programmatic* transaction by autowiring a `'TransactionTemplate'`

TRANSACTIONS – ISOLATION LEVELS

- Transactions are committed at the end of the transaction
 - Changes made in the transaction are already applied to the database, but they are only made permanent at the end of the transaction
 - The save is cancelled when a rollback occurs, and all changes are removed
 - You can compare this with working in a code editor and explicitly saving your changes when you are done. Other can still see you work-in-progress changes before you save.
- But what happens if multiple transactions are running at the same time?
 - Do transactions see each other's work in progress?
 - Depends on the isolation level!

TRANSACTIONS – ISOLATION LEVELS



Without isolation, the transaction from request 2 reads an incorrect result!

This is called a *dirty read*.

TRANSACTIONS – ISOLATION LEVELS

- 4 isolation levels can be used
 - READ_UNCOMMITTED
 - READ_COMMITTED
 - REPEATABLE_READ
 - SERIALIZABLE
- Some DBMS do not support all isolation levels
- In general, isolation levels are a complicated topic
 - DBMS all have differences in the way their isolation policies are implemented
 - This is just to give you some insight!

TRANSACTIONS – READ_UNCOMMITTED

- Lowest isolation level
- Allows dirty reads
- Current transaction can see the result of another uncommitted unit-of-work

@Transactional(isolation = Isolation.*READ_UNCOMMITTED*)

TRANSACTIONS – READ_COMMITTED

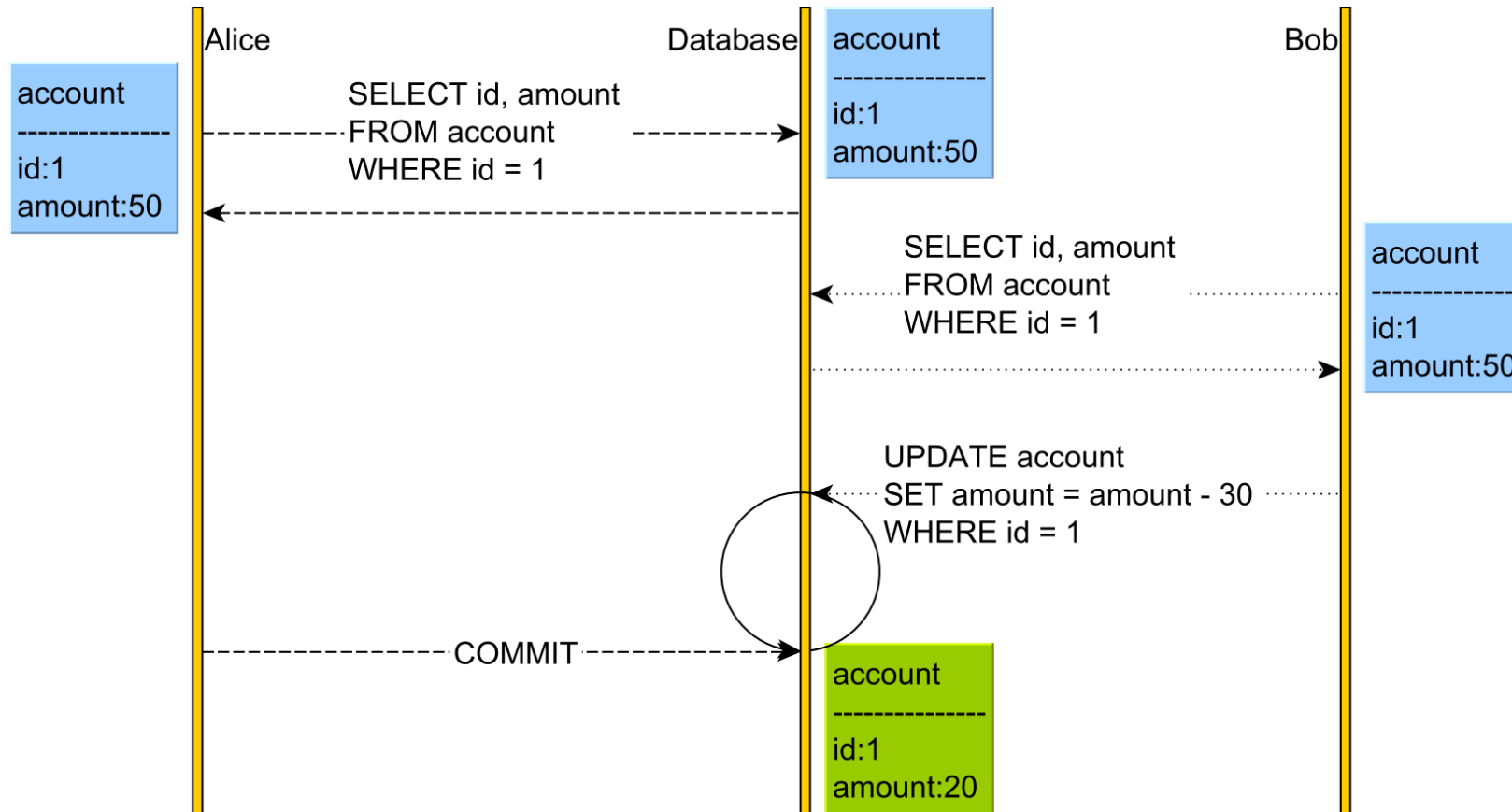
- Does not allow dirty-reads
 - Only committed information can be accessed
- Default strategy for most databases

@Transactional(isolation = Isolation.*READ_COMMITTED*)

TRANSACTIONS – ISOLATION LEVELS

- Just to be complete, two other isolation levels with even higher isolation exist:
 - REPEATABLE_READ
 - (Prevents dirty-reads and non-repeatable reads)
 - SERIALIZABLE
 - (Prevents dirty-reads, non-repeatable and phantom-reads)
- For details on these isolation levels, we refer to the database courses
- The higher the isolation level, the lower the performance. Isolating the transactions has a cost.
 - Isolating transactions usually happens through *locking*
 - Locking allows multiple operations to run concurrently, in isolation
 - The following slides give some **optional** background information on how locking can be accomplished by the database

TRANSACTIONS – PESSIMISTIC LOCKING (OPTIONAL!)

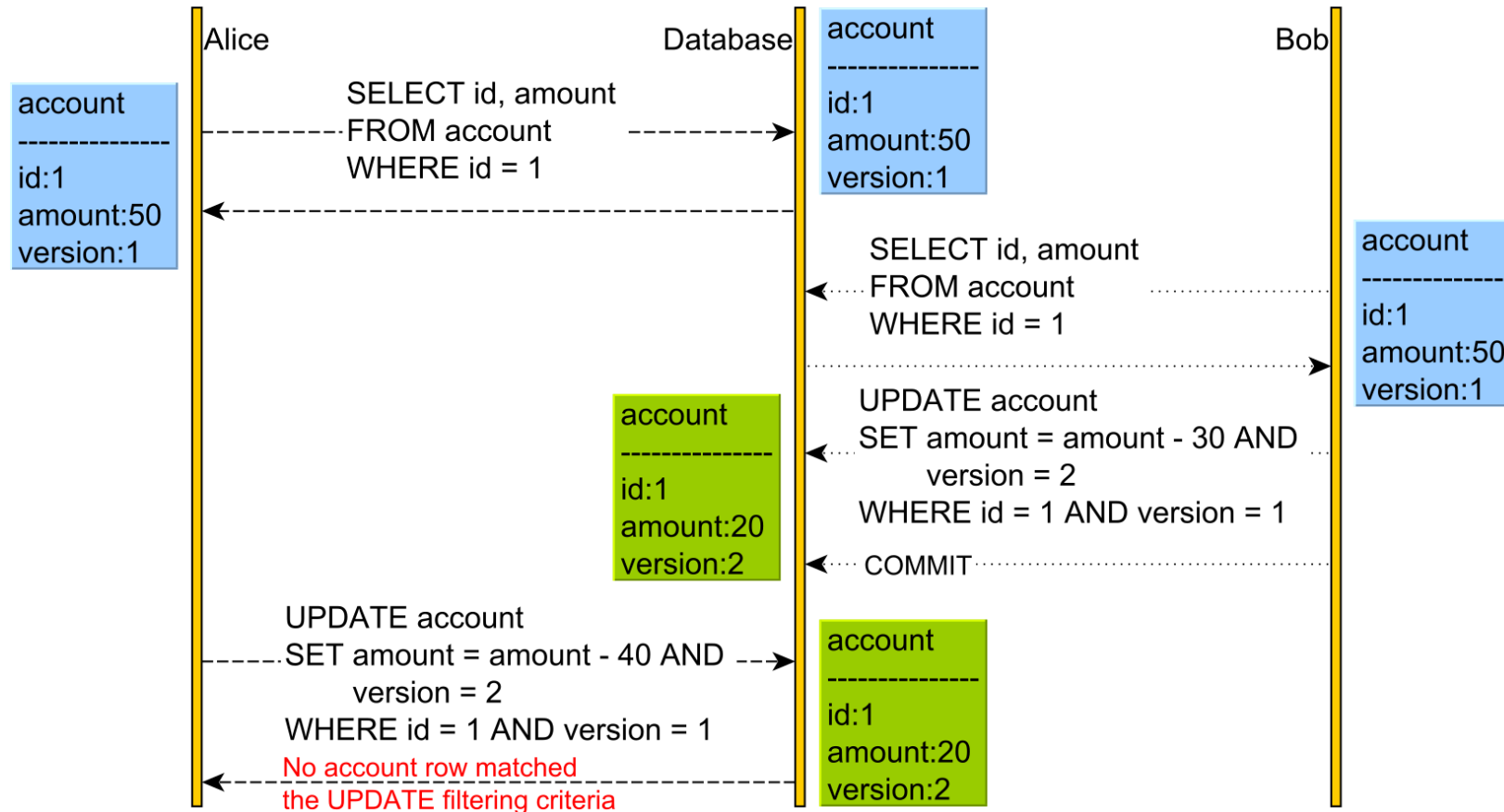


From <https://vladmihalcea.com/optimistic-vs-pessimistic-locking/>

TRANSACTIONS – PESSIMISTIC LOCKING (OPTIONAL!)

- Pessimistic Locking
 - General Idea
 - 'Prevent conflicts from happening'
 - Approach
 - When a connection wants to change a row, it takes an exclusive lock on row(s)
 - Row(s) in the database can only be modified by the connection that holds the lock
 - After the update, the lock is released
 - If another connection tries to change a locked row, an error is thrown
 - Pros and Cons
 - High isolation, but low performance. The cost of obtaining locks is paid by every update, even if there is no risk of a conflict. Roll-backs never occur, since an invalid change is prevented.
 - When to use
 - If there is a high number of conflicts, then this locking strategy has the highest performance, since the cost of roll-backs is avoided.

TRANSACTIONS – OPTIMISTIC LOCKING (OPTIONAL!)



From <https://vladmihalcea.com/optimistic-vs-pessimistic-locking/>

TRANSACTIONS – OPTIMISTIC LOCKING (OPTIONAL!)

- Optimistic Locking
 - General Idea
 - 'Detect and handle conflicts when they occur'
 - Approach
 - Every record has a version number that is incremented on update
 - On update, the connection checks that the number is the same as before it tried to update
 - If this is the case, the update succeeds. Otherwise, an error is thrown.
 - Pros and Cons
 - There is no overhead of obtaining and releasing a lock
 - A conflict is more expensive since the database needs to roll-back a change
 - When to use
 - If there is a low number of conflicts, then this locking strategy has the highest performance. The high cost of roll-back when a conflict occurs is less costly than obtaining a lock on every update.

TRANSACTIONS – PROPAGATION

```
@Service
@Transactional
public class LecturerService {

    // ...

    public Lecturer addLecturer(LecturerInput lecturerInput) {
        var user = userService.signup(Role.LECTURER, lecturerInput.user());
        return lecturerRepository.save(new Lecturer(lecturerInput.expertise(), user));
    }
}
```

```
@Service
@Transactional
public class UserService {

    // ...

    public User signup(Role role, UserInput userInput) {
        // ...
    }
}
```

- What should happen in this case?
 - Should everything run in the same transaction?
 - Or should everything run in its own transaction?

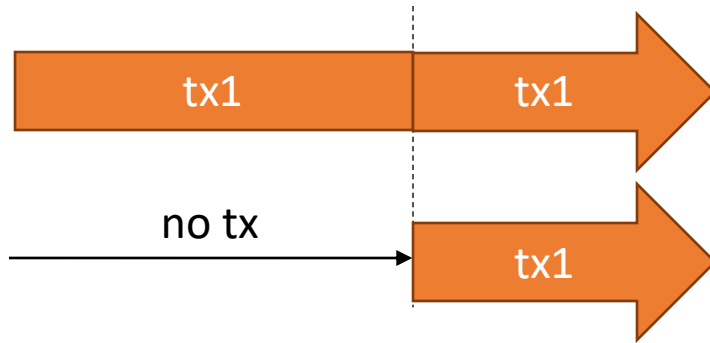
TRANSACTIONS – PROPAGATION

- 7 levels of propagation
- We will show REQUIRED and REQUIRES_NEW
 - Remember that other levels exist should the need ever arise
- Can be configured in the @Transactional annotation

@Transactional(propagation = Propagation.*REQUIRED*)

TRANSACTIONS – REQUIRED

- REQUIRED
 - Default value
 - Execute within a current transaction, create a new one if none exists



@Transactional(propagation = Propagation.*REQUIRED*)

TRANSACTIONS – PROPAGATION

```
@Service
@Transactional
public class LecturerService {

    // ...

    public Lecturer addLecturer(LecturerInput lecturerInput) {
        var user = userService.signup(Role.LECTURER, lecturerInput.user());
        return lecturerRepository.save(new Lecturer(lecturerInput.expertise(), user));
    }
}
```

```
@Service
@Transactional(propagation = Propagation.REQUIRED)
public class UserService {

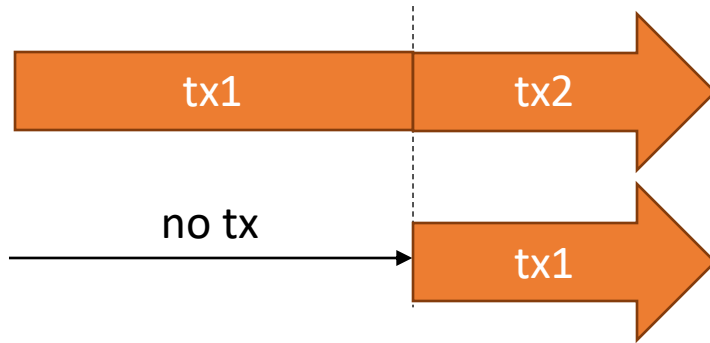
    // ...

    public User signup(Role role, UserInput userInput) {
        // ...
    }
}
```

- If **@Transactional** is present on the LecturerService, the UserService participates in that transaction
- If **@Transactional** is not present on the LecturerService, the UserService creates a new transaction that commits before returning to the LecturerService

TRANSACTIONS – REQUIRES_NEW

- **REQUIRES_NEW**
 - Create a new transaction, suspending the current transaction if one exists



`@Transactional(propagation = Propagation.REQUIRES_NEW)`

TRANSACTIONS – PROPAGATION

```
@Service
@Transactional
public class LecturerService {

    // ...

    public Lecturer addLecturer(LecturerInput lecturerInput) {
        var user = userService.signup(Role.LECTURER, lecturerInput.user());
        return lecturerRepository.save(new Lecturer(lecturerInput.expertise(), user));
    }
}
```

```
@Service
@Transactional(propagation = Propagation.REQUIRES_NEW)
public class UserService {

    // ...

    public User signup(Role role, UserInput userInput) {
        // ...
    }
}
```

- The UserService always creates a new, separate transaction that commits before code goes back to the LecturerService, irrespective of whether the LecturerService created a transaction or not

Transactions

Entity Lifecycle

Cascades

Fetch Types

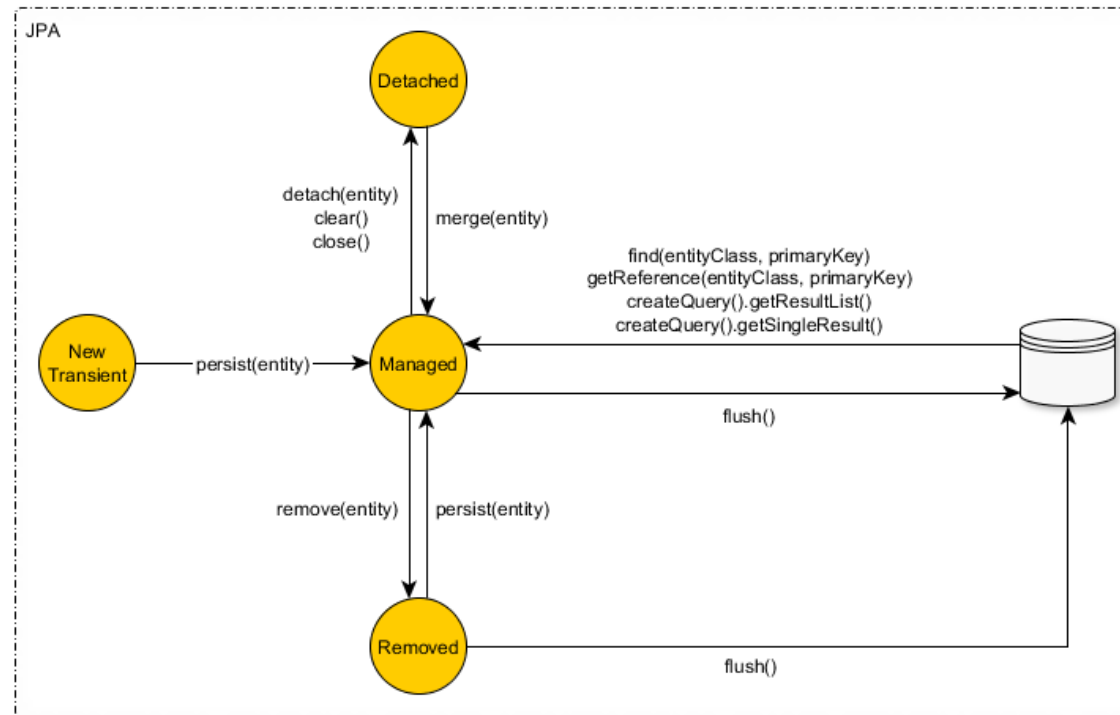
Orphan Removal

Conclusions

Entity Lifecycle

ENTITY LIFECYCLE

- The persistence context is flushed to the DB when the transaction commits
- An entity can be in different states:



From <https://vladmihalcea.com/wp-content/uploads/2014/07/jpaentitystates.png>

ENTITY LIFECYCLE

- Managed
 - Entity is being tracked, its changes will be synced to the DB on flush
- Transient
 - Unknown entity, will be saved as a new row in the DB on flush after being added with persist
- Removed
 - Entity to be deleted, will be removed from the DB on flush
- Detached
 - Disconnected from the persistence context
 - After the persistence context closes, all entities become detached

ENTITY LIFECYCLE – RELATIONSHIPS

```
@Service
@Transactional
public class LecturerService {

    // ...

    public Lecturer addLecturer(LecturerInput lecturerInput) {
        var user = userService.signup(Role.LECTURER, lecturerInput.user());
        return lecturerRepository.save(new Lecturer(lecturerInput.expertise(), user));
    }
}
```

```
@Service
@Transactional
public class UserService {

    public User signup(Role role, UserInput userInput) {

        // ...

        final var user = new User(
            userInput.username(),
            userInput.firstName(),
            userInput.lastName(),
            userInput.email(),
            hashedPassword,
            role
        );

        return userRepository.save(user);
    }
}
```

- What is the result of this code?
 - User is transient, becomes managed after 'save'
 - Lecturer is transient and references a managed instance
 - After the last save, both are managed and saved in the database

ENTITY LIFECYCLE – RELATIONSHIPS

```
@Service
public class LecturerService {

    // ...

    public Lecturer addLecturer(LecturerInput lecturerInput) {
        var user = userService.signup(Role.LECTURER, lecturerInput.user());
        return lecturerRepository.save(new Lecturer(lecturerInput.expertise(), user));
    }
}
```

```
@Service
public class UserService {

    public User signup(Role role, UserInput userInput) {

        // ...

        final var user = new User(
            userInput.username(),
            userInput.firstName(),
            userInput.lastName(),
            userInput.email(),
            hashedPassword,
            role
        );

        return user;
    }
}
```

- What is the result of this code?
 - Lecturer is transient and references a transient instance
 - On save, we get a crash saying that a transient instance cannot reference another transient instance on save: *Not-null property references a transient value - transient instance must be saved before current operation: Lecturer.user -> model.User*

ENTITY LIFECYCLE – UPDATE

```
@Service
public class PonyService {

    private PonyRepository ponyRepository;

    @Autowired
    public PonyService(PonyRepository ponyRepository) {
        this.ponyRepository = ponyRepository;
    }

    public Pony updatePony(String name, Pony newInformation) {
        Pony pony = ponyRepository.findPonyByName(name);
        pony.updateNameAndAge(newInformation.getName(), newInformation.getAge());
        return ponyRepository.save(pony);
    }
}
```

- What is the result of this code?
 - Without transactional, every repository interaction has its own transaction
 - The persistence context remains open during the whole request
 - The updated pony is in the database after the transaction from save has caused a flush to database

ENTITY LIFECYCLE – UPDATE

```
@Service
public class PonyService {

    private PonyRepository ponyRepository;

    @Autowired
    public PonyService(PonyRepository ponyRepository) {
        this.ponyRepository = ponyRepository;
    }

    public Pony updatePony(String name, Pony newInformation) {
        Pony pony = ponyRepository.findPonyByName(name);
        pony.updateNameAndAge(newInformation.getName(), newInformation.getAge());
        return pony;
    }
}
```

- What is the result of this code?
 - The persistence context remains open during the whole request
 - Without transactional, every repository interaction has its own transaction
 - There is no transaction flush after updating the pony since there is no call to save, the row in the database is not updated

ENTITY LIFECYCLE – UPDATE

```
@Service
@Transactional
public class PonyService {

    private PonyRepository ponyRepository;

    @Autowired
    public PonyService(PonyRepository ponyRepository) {
        this.ponyRepository = ponyRepository;
    }

    public Pony updatePony(String name, Pony newInformation) {
        Pony pony = ponyRepository.findPonyByName(name);
        pony.updateNameAndAge(newInformation.getName(), newInformation.getAge());
        return pony;
    }
}
```

- What is the result of this code?
 - The persistence context remains open during the whole request
 - The service-method runs in one transaction
 - The findPonyByName method adds the pony to the persistence context
 - When the transaction ends, the persistence context is flushed to the database
 - At the end of the transaction, the database holds the updated pony without an explicit save

ENTITY LIFECYCLE

- With knowledge of the persistence context, transactions and the entity lifecycle, we can fully explain the behaviour of Spring Data JPA
- General rules
 - Always work in transactions to correctly scope your database access!
 - If a relationship references transient instances, save them first!
- We can simplify some operations by using *cascades*

Transactions

Entity Lifecycle

Cascades

Fetch Types

Orphan Removal

Conclusions

Cascades

CASCADES

- When in a transaction, “pass” or “propagate” an operation on one entity to an associated entity
- Cascade types
 - PERSIST
 - MERGE
 - REMOVE
 - REFRESH
 - DETACH
 - ALL

CASCADES – PERSIST EXAMPLE

```
@OneToOne(cascade = CascadeType.PERSIST)  
@JoinColumn(name = "user_id")  
@NotNull  
private User user;
```

- When the entity containing this field is persisted, then persist the referenced entity as well
- Using cascades can limit the number of needed repository interactions
- Since cascades are specified on the relationship-annotation, they are valid **all the time**

Transactions

Entity Lifecycle

Cascades

Fetch Types

Orphan Removal

Conclusions

Fetch Types

FETCH TYPES

- There are two fetch types

- EAGER

`@OneToOne(fetch = FetchType.EAGER)`

- Immediately retrieve the associated entity from the database

- LAZY

`@OneToOne(fetch = FetchType.LAZY)`

- Only retrieve the associated entity when it is accessed
 - Only works if the entity is managed

- When to use which fetch type?

- Broadly speaking, relationships to a single entity are often eagerly fetched, whereas relationships to multiple entities are usually lazily fetched, since the number of related records to retrieve is not known in advance.
 - [This link](#) provides a more nuanced answer than the broad guideline above

FETCH TYPES

- In JPQL the difference can be made using *fetch*

```
public List<Schedule> findByLecturer_User_Username(String username) {  
    return entityManager  
        .createQuery("""  
            select s from Schedule s  
            join s.lecturer l  
            join l.user u  
            where u.username = :username""", Schedule.class)  
        .setParameter("username", username)  
        .getResultList();  
}
```

Only retrieve schedules, do not retrieve
lecturers and users

```
public List<Schedule> findByLecturer_User_Username(String username) {  
    return entityManager  
        .createQuery("""  
            select s from Schedule s  
            join fetch s.lecturer l  
            join fetch l.user u  
            where u.username = :username""", Schedule.class)  
        .setParameter("username", username)  
        .getResultList();  
}
```

Retrieve schedules and eagerly
retrieve lecturers and users

Transactions

Entity Lifecycle

Cascades

Fetch Types

Orphan Removal

Conclusions

Orphan Removal

ORPHAN REMOVAL

- Can be used on One-to-One and One-to-Many relationships
 - When an entity is removed from the relationship in java-code then the removed entity is automatically deleted in the database
- Removing the relationship...
 - With orphanRemoval: breaks the relationship and **deletes** the referenced entity
 - Without orphanRemoval: breaks the relationship and **keeps** the referenced entity
- What you want depends on your domain model

Transactions

Entity Lifecycle

Cascades

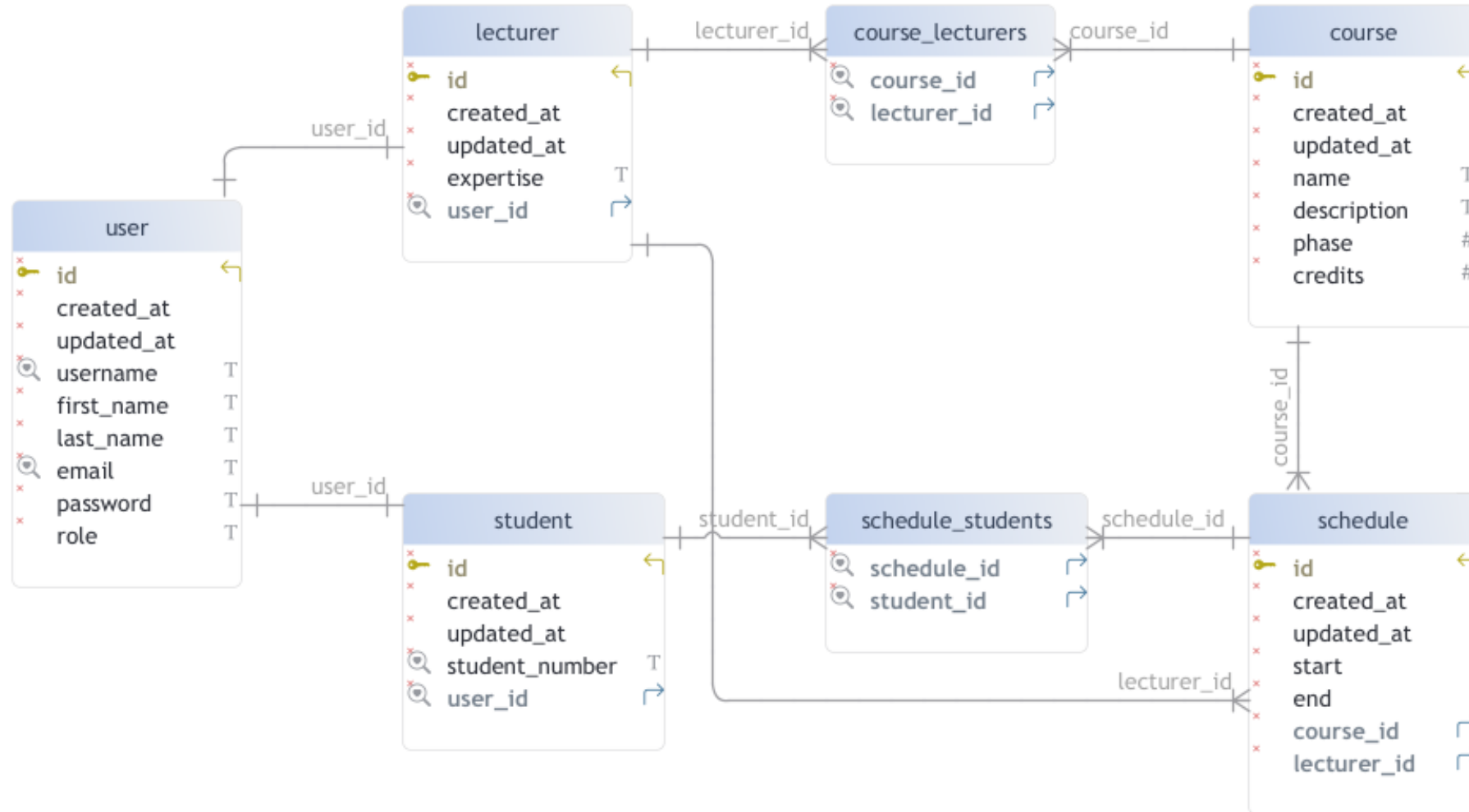
Fetch Types

Orphan Removal

Conclusions

Conclusions

CONCLUSIONS



CONCLUSIONS

- Using JPA, we can perfectly model this domain model
 - We can use transactions, cascades, fetch types and orphanRemoval to make our life as simple as possible
- As the domain model increases, the lack of precise control of JPA can become an issue
 - Queries are being executed all over the place, often in a very inefficient way
 - Relationship properties like cascades or fetch types are often useful in one use-case but bothersome in another one

CONCLUSIONS

- Introducing hard-boundaries in the domain model can counteract the lack of control that JPA introduces
- To manage the complexity of big domain models, aggregates are often introduced:
 - Each *aggregate* is a cluster of objects that 'belong together'
 - They are treated as a single unit when making changes to the data
 - Within an aggregate, objects have references to each other
 - Entities from different aggregates only reference each other by PK
 - This division helps to manage the complexity of both JPA and JDBC
 - The choice of JDBC/JPA can then be made for every aggregate, depending on complexity and use-case
- If you are interested in this topic, this will be continued in the course 'Software Craftmanship'

CONCLUSIONS

- In general
 - Use JDBC if full-control is necessary and/or the domain model is very simple
 - Use JPA in all other cases
 - Or Spring Data JPA when working with JPA and Spring
- Unfortunately... both technologies can be very hard to work with if the domain model grows in size and complexity
 - Splitting-up the domain model in aggregates can help tackle this, but comes with its own complexity

